Scenariusz wystąpienia

Apache POI i Apache Commons CSV · Programowanie zaawansowane, laboratorium 4 · Igor Krzystek, Filip Kontek · wersja dla prezentacji 14-slajdowej

Jak korzystać z tego dokumentu

Dla każdego slajdu jest: kto mówi, ile czasu, po co jest slajd, co powiedzieć i na co uważać. Tekst w niebieskiej ramce to propozycja wypowiedzi. Nie uczcie się go słowo w słowo: przeczytajcie kilka razy i mówcie własnymi słowami. Czas liczony przy tempie ok. 120–130 słów na minutę.

Łączny czas: **ok. 17:55 min** plus pytania. Osoba 1 (CSV) mówi ok. 9:55, Osoba 2 (Excel) ok. 8:00. Podział osób zostawiamy wam. Wymagania nie podają długości referatu; jeśli prowadzący poda limit, użyjcie sekcji „Jak skrócić albo wydłużyć” na końcu.

Harmonogram

Nr	Slajd	Kto mówi	Czas	Koniec o
1	Apache POI i Apache Commons CSV (tytuł)	Osoba 1	0:40	0:40
2	Te same dane, dwa różne formaty	Osoba 1	1:45	2:25
3	Obie biblioteki dodajemy z Mavena	Osoba 1	0:45	3:10
4	Odczyt CSV do listy obiektów	Osoba 1	1:30	4:40
5	CSVFormat - dialekt pliku	Osoba 1	1:00	5:40
6	Uszkodzone i błędne pliki	Osoba 1	1:45	7:25
7	Zapis CSV (cudzysłowy dodaje CSVPrinter)	Osoba 1	1:00	8:25
8	POI - odczyt .xlsx do listy rekordów	Osoba 2	1:45	10:10
9	Daty to liczby, formuły to przepis	Osoba 2	1:30	11:40
10	Zapis .xlsx: style tworzymy raz	Osoba 2	1:15	12:55
11	Duże pliki: SXSSF trzyma w pamięci tylko okno	Osoba 2	1:15	14:10
12	Demo: konwerter CSV → Excel	razem	2:30	16:40
13	Dobre praktyki	razem	1:00	17:40
14	Dziękujemy za uwagę	Osoba 2	0:15	17:55
	Razem		**17:55**	

Zgodność z wymaganiami (referaty.pdf, temat 4)

Wymaganie	Gdzie
Prezentacja na jasnym tle	Wszystkie slajdy
Referat 2- lub 3-osobowy	2 osoby: Osoba 1 (CSV), Osoba 2 (Excel/POI)
Cel: odczyt i zapis .xlsx przez POI (XSSF / SXSSF)	Slajdy 8–11, demo 12
Cel: odczyt i zapis CSV z Commons CSV (delimitery, nagłówki)	Slajdy 4–7, demo 12
Cel: różnice, pułapki, dobre praktyki	Slajdy 2, 5–7, 9–11, 13
Cel: pliki → obiekty Javy (DTO/record) i z powrotem	Slajdy 2, 4, 7, 8, 10
1) Excel vs CSV, reguła kciuka	Slajd 2 (**reguła kciuka tylko na głos**)
2) Maven, zależności	Slajd 3
3) POI odczyt: Workbook→Sheet→Row→Cell, DataFormatter, record Person, getLocalDateTimeCellValue()	Slajdy 8–9 (**struktura Workbook→Sheet→Row→Cell tylko na głos**)
4) POI zapis, SXSSF, precyzja pieniędzy, >100k wierszy	Slajdy 10–11
5) CSV odczyt: nagłówki, separatory, kodowanie, cytowania, dialekty, błędy, locale, streaming, sanity check, pułapki	Slajdy 4–6
6) CSV zapis: delimiter, nagłówki jawnie	Slajd 7
7) Dobre praktyki i pułapki	Slajd 13

Dwa punkty z wymagań nie mają osobnego miejsca na slajdach tej wersji, dlatego są w scenariuszu do powiedzenia na głos: reguła kciuka na slajdzie 2 i struktura Workbook → Sheet → Row → Cell na slajdzie 8. Wymagania mówią też, że referat trzeba oddać zgodnie z instrukcją do laboratorium i w terminie (inaczej ocena spada o 50–100%). Sprawdźcie w instrukcji, jak i gdzie go oddać.

Przygotowanie przed zajęciami

- **Podmieńcie w repozytorium trzy pliki konwertera**: przyklad/src/main/java/Main.java, przyklad/pom.xml i przyklad/dane.csv (wysłane razem z tym PDF-em).
- **Wersja Javy:** pom.xml wymaga teraz Javy 17 lub nowszej (wcześniej 25). Na komputerze do prezentacji sprawdźcie `java -version`.
- **Zbudujcie i uruchomcie demo** na tym samym komputerze: w katalogu przyklad `mvn clean package`, potem `java -jar target/przyklad-1.0-SNAPSHOT-jar-with-dependencies.jar`. Program musi wypisać „Utworzono plik wynik.xlsx, wierszy danych: 4”. Nie mogłem pobrać bibliotek z Maven Central, więc Main.java jest sprawdzony kompilatorem tylko na atrapach API; pierwsze prawdziwe uruchomienie róbcie wcześniej, nie na zajęciach.
- Uruchomcie też przykłady kodu ze slajdów 4, 6, 7, 8, 9, 10 i 11. Nie były kompilowane z prawdziwymi bibliotekami.
- Zachowajcie gotowy wynik.xlsx jako plan B i sprawdźcie, czym go otworzycie (Excel albo LibreOffice).
- Otwórzcie wcześniej IntelliJ z Main.java i terminal w katalogu przyklad; powiększcie czcionki.
- Zróbcie jedną pełną próbę na głos ze stoperem. Ustalcie sygnał przekazania głosu (slajd 7 → 8).

Scenariusz slajd po slajdzie

Slajd 1 · Apache POI i Apache Commons CSV (tytuł)

Mówi: **Osoba 1** · Czas: **0:40** · od 0:00 do 0:40

Po co ten slajd: Przedstawić się, temat i plan.

Co powiedzieć

> Dzień dobry, nazywamy się Igor Krzystek i Filip Kontek. Nasz referat dotyczy dwóch bibliotek z rodziny Apache: Apache POI, która służy do pracy z plikami Excela, oraz Apache Commons CSV do plików CSV.
>
> Pokażemy, jak w Javie czytać i zapisywać takie pliki i jak zamieniać ich zawartość na obiekty. Najpierw część o CSV, potem o Excelu, a na końcu demo z naszego repozytorium. Arkusz, który widzicie po prawej, wygeneruje właśnie to demo.

Wskazówki

- Nie czytaj tytułu ze slajdu, publiczność go widzi.
- Osoba 2 stoi obok i włącza się od slajdu 8.

Przejście: „Zacznijmy od tego, po co w ogóle są te biblioteki.”

Slajd 2 · Te same dane, dwa różne formaty

Mówi: **Osoba 1** · Czas: **1:45** · od 0:40 do 2:25

Po co ten slajd: Pokazać problem, porównać formaty i podać regułę wyboru (punkt 1 zakresu).

Co powiedzieć

> U góry ten sam zbiór danych w dwóch formatach. Po lewej CSV otwarty w notatniku: to zwykły tekst, wartości oddziela średnik, pensja 15000,50 jest napisem, a adres zawiera średnik w środku, więc musi być ujęty w cudzysłów. Po prawej to samo w Excelu: komórki mają typy i formaty, kwota wyświetla się jako złotówki. Ale plik .xlsx to w środku archiwum ZIP z plikami XML, więc bez biblioteki nie odczytamy nawet jednej komórki.
>
> Tabela niżej zbiera różnice. CSV nie ma typów, wszystko jest Stringiem, ale jest lekki i można go czytać wiersz po wierszu. Jego problemy to separator, kodowanie i cudzysłowy. Excel ma typy, formuły i formatowanie, ale standardowo trzyma cały plik w pamięci, a daty i formuły potrafią zaskoczyć.
>
> Stąd reguła kciuka: jeśli dane idą z systemu do systemu, na przykład eksport z bazy czy proces ETL, wybieramy CSV. Jeśli plik ma otworzyć człowiek, na przykład księgowość, generujemy Excela.
>
> W obu przypadkach program chce dostać to samo, czyli listę obiektów, na przykład rekordów Pracownik z pensją jako BigDecimal. O tym, jak do tego dojść, jest cały referat.

Wskazówki

- **Regułę kciuka (ETL → CSV, raporty dla ludzi → Excel) powiedzcie na pewno.** Jest wprost w wymaganiach, a na slajdzie jej nie ma.
- Pokaż średnik wewnątrz cudzysłowu w adresie: wrócimy do niego na slajdzie 6.
- Tabeli nie czytaj wiersz po wierszu, wybierz 2–3 wiersze (Typy danych, Pamięć).

Przejście: „Żeby z nich skorzystać, trzeba je najpierw dodać do projektu.”

Slajd 3 · Obie biblioteki dodajemy z Mavena

Mówi: **Osoba 1** · Czas: **0:45** · od 2:25 do 3:10

Po co ten slajd: Zależności Maven (punkt 2 zakresu) i najczęstszy błąd.

Co powiedzieć

> Żadna z tych bibliotek nie jest częścią JDK, więc dodajemy je w pom.xml. Wersje trzymamy w properties.
>
> Uwaga na częsty błąd: do plików .xlsx potrzebujemy artefaktu poi-ooxml, a nie samego poi. Samo poi obsługuje tylko stary format .xls. poi-ooxml sam dociąga resztę zależności. Commons CSV to jeden artefakt.
>
> Nasz projekt budujemy jako fat JAR, więc biblioteki są w środku JAR-a i wystarczy zwykłe java -jar.

Wskazówki

- Na pytanie o nowsze wersje: są, ale API, którego używamy, się nie zmieniło.

Przejście: „Mamy biblioteki, to czytamy pierwszy plik CSV."

Slajd 4 · Odczyt CSV do listy obiektów

Mówi: **Osoba 1** · Czas: **1:30** · od 3:10 do 4:40

Po co ten slajd: Pełny odczyt CSV z mapowaniem na rekord (punkt 5 zakresu, cel: DTO/record).

Co powiedzieć

> W praktyce potrzebujemy czterech klas, są wypisane po prawej. CSVFormat opisuje, jak wygląda plik. Bierzemy domyślny format i builderem zmieniamy separator na średnik, mówimy, że pierwsza linia to nagłówki, i każemy pomijać puste linie.
>
> Plik otwieramy przez Files.newBufferedReader z jawnie podanym UTF-8, w try-with-resources, żeby zamknął się nawet przy wyjątku. format.parse daje CSVParser, który jest Iterable, więc idziemy zwykłą pętlą for.
>
> Każdy CSVRecord to jeden wiersz. Dzięki nagłówkom czytamy kolumny po nazwie: get("Imie"), get("Stanowisko"). Pensję zamieniamy na BigDecimal, po drodze zamieniając przecinek na kropkę, bo tak zapisuje polski Excel. Od razu tworzymy obiekt Pracownik, więc reszta programu nie widzi już pliku, tylko obiekty. Czwarta klasa, CSVPrinter, służy do zapisu; wrócę do niej za chwilę.

Wskazówki

- To najważniejszy slajd części CSV: idź po kodzie z góry na dół, nie spiesz się.
- Na pytanie o duży plik: pętla czyta rekord po rekordzie, więc pamięć nie rośnie z rozmiarem pliku. Rośnie tylko nasza lista; przy ogromnych plikach przetwarzamy rekord od razu, bez listy.

Przejście: „Ustawiliśmy kilka opcji formatu. Zobaczmy, jakie są możliwości."

Slajd 5 · CSVFormat - dialekt pliku

Mówi: **Osoba 1** · Czas: **1:00** · od 4:40 do 5:40

Po co ten slajd: Dialekty i konfiguracja parsera: separatory, nagłówki.

Co powiedzieć

> CSV to nie jeden format, tylko rodzina dialektów. Commons CSV ma gotowe stałe z lewej tabeli: DEFAULT to rozsądny punkt startowy, RFC4180 trzyma się standardu, EXCEL odpowiada eksportowi z Excela, a TDF to pliki rozdzielane tabulatorem.
>
> Każdą stałą dostrajamy builderem, najważniejsze opcje są po prawej: separator, nagłówki z pliku albo podane w kodzie, pominięcie pierwszej linii, obcinanie spacji i zamiana pustych pól na null. Polski Excel zapisuje CSV ze średnikiem, więc na labach prawie zawsze zmienimy separator.
>
> CSVFormat jest niezmienny, builder tworzy kopię. Dlatego format definiujemy raz, jako stałą, i używamy tej samej stałej do odczytu i zapisu.

Wskazówki

- Jeśli jest czas: setHeader("A", "B") przy pliku z linią nagłówka bez setSkipHeaderRecord(true) wczyta nagłówek jako pierwszy rekord danych.

Przejście: „Wszystko działa, dopóki plik jest poprawny. A rzadko jest.”

Slajd 6 · Uszkodzone i błędne pliki

Mówi: **Osoba 1** · Czas: **1:45** · od 5:40 do 7:25

Po co ten slajd: Obsługa braków i błędów, sanity check, pułapki CSV.

Co powiedzieć

> Pliki z prawdziwego świata bywają zepsute, więc nie ufamy im na ślepo. Po otwarciu parsera sprawdzamy, czy są wszystkie potrzebne kolumny, przez getHeaderMap().containsKey. Jeśli którejś brakuje, rzucamy wyjątek z nazwą kolumny, zamiast wywrócić się w połowie pliku z niejasnym komunikatem.
>
> W pętli isConsistent sprawdza, czy rekord ma tyle pól, ile nagłówek, a isBlank wyłapuje puste wartości. W komunikacie podajemy numer rekordu, żeby dało się go znaleźć w pliku. Przecinek dziesiętny normalizujemy albo parsujemy przez NumberFormat dla polskiego Locale.
>
> Na dole trzy rzeczy. Sanity check: przy dużym pliku najpierw wczytujemy kilka rekordów, na przykład dziesięć, i wypisujemy ich liczbę i unikalne wartości jednej kolumny. Jeśli zobaczymy jedną długą kolumnę ze średnikami w środku, mamy zły separator. Wielkość liter: Salary i salary to dla parsera dwie różne kolumny. I split kontra parser: split(";") rozbije adres „ul. Długa 5; m. 3” ze slajdu 2 na dwa pola i przesunie wszystkie kolumny, a parser wie, że tekst w cudzysłowie to jedno pole.

Wskazówki

- Sanity check jest wprost w wymaganiach, wymień go wyraźnie.
- Wynik split dla linii z adresem sprawdziliśmy w Javie: 4 pola zamiast 3.

Przejście: „Wiemy, jak czytać. Teraz zapis.”

Slajd 7 · Zapis CSV (cudzysłowy dodaje CSVPrinter)

Mówi: **Osoba 1** · Czas: **1:00** · od 7:25 do 8:25

Po co ten slajd: Zapis CSV (punkt 6 zakresu): delimiter, jawne nagłówki, BOM; przekazanie głosu.

Co powiedzieć

> Zapis to lustrzane odbicie odczytu. Budujemy format z tym samym separatorem, ale nagłówki podajemy jawnie.
>
> Otwieramy Writer z UTF-8 i na samym początku zapisujemy BOM, czyli niewidoczny znak U+FEFF. Dzięki niemu Excel na Windowsie rozpozna UTF-8 i poprawnie pokaże polskie znaki. BOM musi być przed nagłówkiem, a CSVPrinter wypisuje nagłówek już w konstruktorze, dlatego piszemy go wcześniej. Potem printRecord dla każdego pracownika.
>
> Po prawej wynik: pole „Nowak; Anna” zawiera średnik, więc CSVPrinter sam ujął je w cudzysłów. BigDecimal zapisze się z kropką, a polski Excel może to wziąć za tekst, więc plik dla człowieka lepiej od razu zrobić jako .xlsx.
>
> To tyle o CSV. Teraz o Excelu i Apache POI opowie [imię Osoby 2].

Wskazówki

- **Uwaga do kodu na slajdzie:** w linii w.write('') między apostrofami jest niewidoczny znak BOM, więc wygląda to jak pusty znak. Powiedz na głos: „tu zapisujemy znak \uFEFF”. Jeśli ktoś przepisze kod ze slajdu, niech wpisze w.write('\uFEFF').
- Pokaż na podglądzie wyniku cudzysłów wokół „Nowak; Anna”.
- Wyraźnie oddaj głos Osobie 2.

Przejście: Osoba 2 przejmuje.

Slajd 8 · POI - odczyt .xlsx do listy rekordów

Mówi: **Osoba 2** · Czas: **1:45** · od 8:25 do 10:10

Po co ten slajd: Model pliku w POI, odczyt .xlsx, DataFormatter, mapowanie na record Person (punkt 3 zakresu).

Co powiedzieć

> Dzięki. Najpierw jak POI widzi plik Excela: dokładnie tak jak my w programie. Workbook to cały plik, Sheet to zakładka, Row to wiersz, a Cell to komórka. Do tego trzy implementacje: HSSF dla starego .xls, XSSF dla .xlsx i SXSSF do zapisywania bardzo dużych plików.
>
> Rekord Person z imieniem, datą urodzenia i pensją to dokładnie przykład z wymagań. WorkbookFactory.create otwiera plik i sam rozpoznaje, czy to .xls, czy .xlsx. Bierzemy pierwszy arkusz i iterujemy po wierszach, pomijając wiersz nagłówka. Uwaga na numerację: POI liczy od zera, więc wiersz 2 w Excelu to getRow(1), a kolumna B to getCell(1).
>
> Imię czytamy przez DataFormatter, który zwraca tekst dokładnie taki, jak widzi go użytkownik, i nie rzuca wyjątku dla liczby ani dla brakującej komórki. Datę czytamy getLocalDateTimeCellValue, bo to prawdziwa data Excela; zaraz wyjaśnię, czemu to ważne. Pensję bierzemy jako liczbę i zamieniamy na BigDecimal przez valueOf.
>
> Pusta komórka często w ogóle nie istnieje i getCell zwraca null. Wtedy pomaga polityka CREATE_NULL_AS_BLANK.

Wskazówki

- **Strukturę Workbook → Sheet → Row → Cell powiedzcie na pewno.** Jest wprost w wymaganiach (punkt 3), a w tej wersji prezentacji nie ma jej na slajdzie.
- Pełny zapis polityki: row.getCell(i, Row.MissingCellPolicy.CREATE_NULL_AS_BLANK).

Przejście: „Obiecałem wyjaśnić daty."

Slajd 9 · Daty to liczby, formuły to przepis

Mówi: **Osoba 2** · Czas: **1:30** · od 10:10 do 11:40

Po co ten slajd: Daty jako liczby, DataFormatter, FormulaEvaluator.

Co powiedzieć

> Komórka w Excelu pokazuje 1 listopada 2023, a getNumericCellValue zwraca 45231. Excel trzyma datę jako liczbę dni od początku 1900 roku, a to, że jest to data, wynika tylko z formatu komórki. Dlatego datę czytamy getLocalDateTimeCellValue, a tekst taki jak na ekranie daje DataFormatter. DateUtil.isCellDateFormatted mówi, czy komórka jest datą. getStringCellValue na komórce liczbowej rzuca wyjątek.
>
> Po prawej formuły. getCellType zwraca FORMULA, a getNumericCellValue zwraca wynik zapamiętany przez Excela przy ostatnim zapisie pliku. Jeśli zmienimy dane w Javie, ten wynik jest nieaktualny i trzeba przeliczyć formułę przez FormulaEvaluator, tak jak w kodzie na slajdzie.

Wskazówki

- Liczbę 45231 dla 1.11.2023 sprawdziliśmy obliczeniem w Javie.
- Przykład formuły do powiedzenia: komórka D2 z formułą =C2*12, czyli roczna pensja.

Przejście: „To był odczyt. Teraz w drugą stronę."

Slajd 10 · Zapis .xlsx: style tworzymy raz

Mówi: **Osoba 2** · Czas: **1:15** · od 11:40 do 12:55

Po co ten slajd: Zapis .xlsx, style, precyzja pieniędzy (punkt 4 zakresu).

Co powiedzieć

> Zapis idzie odwrotnie niż odczyt: tworzymy skoroszyt, arkusz, wiersze i komórki, a na końcu zapisujemy do strumienia. Skoroszyt i strumień otwieramy w jednym try-with-resources.
>
> Najważniejsze: styl, na przykład format kwoty, jest obiektem całego pliku. Tworzymy go raz, przed pętlą, i tylko przypisujemy komórkom. Jeśli wywołamy createCellStyle w pętli, każda komórka dostanie nowy styl, a .xlsx ma limit 64 tysięcy stylów.
>
> Pieniądze w kodzie trzymamy w BigDecimal, bo double nie zapisze dokładnie ułamków dziesiętnych: 0,1 plus 0,2 daje 0,30000000000000004. Do komórki zapisujemy liczbę z formatem kwoty albo tekst, gdy liczy się każdy grosz. Daty też potrzebują stylu z formatem daty, inaczej zobaczymy surową liczbę 45231.

Wskazówki

- Wynik 0.1 + 0.2 sprawdziliśmy w Javie.

Przejście: „A co, jeśli raport ma milion wierszy?"

Slajd 11 · Duże pliki: SXSSF trzyma w pamięci tylko okno

Mówi: **Osoba 2** · Czas: **1:15** · od 12:55 do 14:10

Po co ten slajd: SXSSF, pamięć, limity (punkt 4 zakresu: duże pliki).

Co powiedzieć

> XSSFWorkbook trzyma każdą komórkę jako obiekt w pamięci i przy setkach tysięcy wierszy program kończy się OutOfMemoryError. Do takich plików jest SXSSFWorkbook, czyli wersja strumieniowa.
>
> W konstruktorze podajemy rozmiar okna, tu 100 wierszy. W pamięci są tylko ostatnie wiersze; gdy tworzymy kolejny, najstarszy trafia do pliku tymczasowego na dysku i nie da się już do niego wrócić. Kod wygląda prawie tak samo jak przy XSSF. Na końcu dispose usuwa pliki tymczasowe.
>
> Arkusz ma limit 1 048 576 wierszy, więc większe dane dzielimy na arkusze. SXSSF służy tylko do zapisu; czytanie ogromnych plików robi się osobnym, zdarzeniowym API.

Wskazówki

- Pokaż na diagramie: szare paski to dysk, niebieskie okno to pamięć, strzałka to nowe wiersze.
- Zasada z wymagań: powyżej ok. 100 tys. wierszy SXSSF, mało stylów, strumienie zamykane od razu.

Przejście: „Teorii wystarczy, pokażemy to w działaniu.”

Slajd 12 · Demo: konwerter CSV → Excel

Mówi: **razem** · Czas: **2:30** · od 14:10 do 16:40

Po co ten slajd: Pokazać obie biblioteki razem na projekcie z repozytorium.

Co powiedzieć

> **Osoba 1 (ok. 1:00):** Teraz pokażemy, jak obie biblioteki działają razem. To projekt z naszego repozytorium: program czyta dane.csv i zapisuje wynik.xlsx. [Pokazuje dane.csv, potem Main.java w IntelliJ.] Krok 1: CSVFormat ze średnikiem i nagłówkami, jak na slajdzie 4. Plik czytamy w UTF-8, a mała metoda pominBom usuwa znak BOM, jeśli plik zapisał Excel. Krok 2: zanim cokolwiek przetworzymy, sprawdzamy, czy są wszystkie kolumny, jak na slajdzie 6.
>
> **Osoba 2 (ok. 1:30):** Krok 3: styl pogrubionego nagłówka tworzymy raz, przed pętlą. Krok 4: wiersz 0 to nagłówek. Krok 5: kod, który widzicie na slajdzie: każdy rekord CSV to nowy wiersz Excela, trzy komórki, a pensja trafia jako liczba, nie tekst. Krok 6: zapis wynik.xlsx w try-with-resources. [Terminal: mvn clean package, potem java -jar …] Program wypisał „Utworzono plik wynik.xlsx, wierszy danych: 4”. [Otwiera plik.] Mamy pogrubiony nagłówek, a pensje są liczbami, wyrównanymi do prawej, więc Excel może na nich liczyć.

Wskazówki

- Komentarze w Main.java są ponumerowane 1–6, idźcie po nich.
- Podgląd wynik.xlsx na slajdzie pokazuje same dane, bez wiersza nagłówka; prawdziwy plik ma w wierszu 1 pogrubiony nagłówek, a dane od wiersza 2. Powiedzcie to przy otwieraniu pliku.
- Plan B: jeśli build się nie uda, otwórzcie wcześniej wygenerowany wynik.xlsx i omówcie kod bez uruchamiania.
- Powiększ czcionkę w IntelliJ (np. 20 pt) i w terminalu.

Przejście: Osoba 1: „Podsumujmy to, co najważniejsze.”

Slajd 13 · Dobre praktyki

Mówi: **razem** · Czas: **1:00** · od 16:40 do 17:40

Po co ten slajd: Podsumowanie dobrych praktyk i pułapek (punkt 7 zakresu).

Co powiedzieć

> **Osoba 1 (CSV, ok. 0:30):** Przy CSV: separator, nagłówki i cytowanie trzymamy w jednej stałej CSVFormat. Kodowanie zawsze jawnie, a BOM, gdy plik otworzy Excel na Windowsie. Cytowanie zostawiamy bibliotece, nigdy split ani String.join. Duże pliki czytamy iteracyjnie, a na starcie sprawdzamy nagłówki i robimy sanity check.
>
> **Osoba 2 (Excel, ok. 0:30):** Przy Excelu: daty przez getLocalDateTimeCellValue albo DataFormatter, formuły przez FormulaEvaluator, style raz na skoroszyt, kwoty w BigDecimal, a powyżej około 100 tysięcy wierszy SXSSF i dispose. I wspólnie: każdy plik w try-with-resources, a zamiana pliku na obiekty w jednym miejscu kodu.

Wskazówki

- Mówcie z głowy, slajd jest ściągą dla słuchaczy.

Przejście: Osoba 2: „To wszystko z naszej strony.”

Slajd 14 · Dziękujemy za uwagę

Mówi: **Osoba 2** · Czas: **0:15** · od 17:40 do 17:55

Po co ten slajd: Zakończyć i otworzyć pytania.

Co powiedzieć

> Dziękujemy za uwagę. Linki do dokumentacji obu bibliotek są na slajdzie, a kod demo jest w naszym repozytorium. Chętnie odpowiemy na pytania.

Wskazówki

- Zostawcie ten slajd na ekranie przez całą rundę pytań.
- Odpowiada ten, czyjej części dotyczy pytanie (CSV: Osoba 1, Excel: Osoba 2).

Konwerter z demo: co robi każdy krok Main.java

Komentarze w kodzie są ponumerowane tak samo jak poniżej.

- **1. Format CSV.** CSVFormat.DEFAULT z separatorem ';', nagłówkami z pierwszej linii, pomijaniem pustych linii i obcinaniem spacji. Plik czytamy w UTF-8; metoda pominBom usuwa niewidoczny znak BOM, który Excel dopisuje przy zapisie „CSV UTF-8”. Bez tego get("Imie") nie znalazłoby kolumny.
- **2. Walidacja nagłówków.** Jeśli brakuje kolumny Imie, Stanowisko albo Pensja, program kończy się czytelnym komunikatem „Brak kolumny w pliku CSV: …”.
- **3. Styl.** Pogrubiona czcionka i styl nagłówka tworzone raz, przed pętlą.
- **4. Wiersz nagłówka.** setHeader() „zużywa” pierwszą linię CSV, więc nagłówek do Excela wpisujemy sami w wierszu 0.
- **5. Dane.** Każdy rekord CSV to nowy wiersz. isConsistent() pilnuje liczby pól. Pensja: przecinek na kropkę, BigDecimal, a do komórki liczba (doubleValue()), więc Excel traktuje ją jako liczbę, a nie tekst.
- **6. Zapis.** Szerokość kolumn na 20 znaków, zapis wynik.xlsx w try-with-resources i komunikat z liczbą wierszy.

Co się zmieniło względem poprzedniej wersji z repo: dodany wiersz nagłówka, pensja jako liczba zamiast tekstu, jawne UTF-8 i obsługa BOM, sprawdzanie kolumn, Java 17 zamiast 25, dane.csv zgodne ze slajdami (pensje 6 i 7).

Pytania, które mogą paść

Czym różni się XSSF od SXSSF?

XSSF trzyma cały arkusz w pamięci i pozwala czytać i zapisywać. SXSSF tylko zapisuje i trzyma w pamięci okno ostatnich wierszy, starsze trafiają do pliku tymczasowego.

Dlaczego nie double dla pieniędzy?

double to liczba binarna i nie zapisze dokładnie wielu ułamków dziesiętnych, np. 0,1. Błędy się sumują. BigDecimal przechowuje wartość dziesiętnie i dokładnie.

Czy Commons CSV poradzi sobie z plikiem kilku GB?

Tak, CSVParser czyta strumieniowo. Warunek: nie wołać getRecords() i nie zbierać rekordów w listę.

Co, jeśli plik CSV nie ma nagłówka?

Czytamy po indeksie, get(0), albo podajemy nazwy w kodzie: setHeader("Imie", "Stanowisko", "Pensja").

Jak przeczytać stary plik .xls?

Tym samym kodem: WorkbookFactory.create wybierze HSSFWorkbook, a my pracujemy na interfejsach.

Jak zapisać formułę?

cell.setCellFormula("C2*12") (bez znaku =). Żeby Excel przeliczył przy otwarciu: wb.setForceFormulaRecalculation(true).

Dlaczego nie napisać parsera CSV samemu?

Cytowania, podwojone cudzysłowy, separator i nowa linia w polu, BOM. Ręczny split psuje dane (slajd 6).

Są alternatywy?

Dla CSV np. OpenCSV albo moduł CSV z Jacksona, dla Excela np. FastExcel. POI jest najbardziej kompletny.

Czemu w konsoli jest ostrzeżenie Log4j2?

POI loguje przez log4j-api i informuje, że nie ma implementacji logowania. Jest nieszkodliwe.

Jak skrócić albo wydłużyć

- **Do ok. 15 minut:** slajd 2 bez czytania tabeli (tylko problem i reguła kciuka), slajd 5 w 30 s (separator i stała), slajd 11 w 45 s, w demo tylko uruchomienie i wynik.
- **Jeśli zostaje czas:** w demo zmieńcie w dane.csv separator na przecinek i pokażcie komunikat o brakującej kolumnie; na slajdzie 9 dodajcie formatCellValue(cell, ev).
- Punkty kontrolne: koniec slajdu 7 (zmiana prelegenta) ok. 8,5 minuty, start demo ok. 14 minuty.